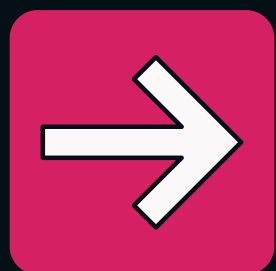


ANGULAR SIGNALS

Getting Started with
Angular Signals.



What are Signals and why do they matter in Angular?



01 WHAT IS A SIGNAL

A Signal is a new reactive primitive introduced in Angular to help manage and track state changes in a simple and efficient way.

Unlike @Input() or RxJS Observable, a signal is:

- ✓ Synchronous
- ✓ Dependency-tracked
- ✓ Fine-grained

*Signals provide a way to read and react to value changes **without** needing a subscription.*



02 SIGNALS IN ACTION

Now that you know what a Signal is, let's look at how they behave in code.



```
import { signal } from '@angular/core';

const counter = signal(0);

// read
console.log(counter()); // output -> 0

// update
counter.set(1);
counter.update(value => value + 1);

console.log(counter()); // output -> 2
```

03 WHY ARE SIGNALS POWERFUL?

You might be wondering: why switch to Signals?



Transparent Reactivity

Angular automatically tracks signal usage and re-renders only where needed.



Simpler State Management

You no longer need external state libraries for many cases.



High Performance

Fine-grained updates reduce unnecessary change detection.

04 DERIVED & EFFECTS

Signals aren't just about storing values. You can also derive new ones and trigger effects when things change.

👉 **computed()** → Create derived state.

👉 **effect()** → Run side-effects when dependencies change



```
const fullName = computed(() =>
    firstName() + ' ' + lastName()
);

effect(() => {
    console.log('Full name changed:', fullName());
});
```

05 WHEN TO USE SIGNALS?

Use Signals when you want to manage local or shared state in a simple, predictable way.

- ✓ UI state inside components
- ✓ Shared state across services
- ✓ Lightweight state without external libraries
- ✗ Keep using RxJS for event streams and async data

*If you love **clean code** and predictable reactivity, Signals are worth trying today!*



ENJOYED IT?
DROP YOUR THOUGHTS
OR TIPS IN THE
COMMENTS!

